

NIMMALA PRANAVI

Role: ai/ml intern

Ai research team

MediaPipe Pose for Virtual Try-On Systems

MediaPipe Pose is a machine learning framework developed by Google that detects 33 human body landmarks in real time. In Virtual Try-On (VTO) systems, it acts as a digital skeleton used to align and place virtual garments on the user's body.

It helps:

- * Track body movement
- * Estimate body posture
- * Align clothing with body landmarks
- * Maintain real-time responsiveness

Working Pipeline of MediaPipe Pose in VTO

1. Detection & Tracking (BlazePose)

MediaPipe Pose uses a two-stage pipeline:

- * A detector first identifies the person in the frame and creates a Region of Interest (ROI).
- * A tracker then processes the ROI continuously and predicts 33 body landmarks such as shoulders, elbows, wrists, hips, knees, and ankles.

This allows stable body tracking across video frames.

2. Body Mapping & Scaling

The detected landmarks are used to calculate body proportions.

Examples:

- * Shoulder width
- * Torso length
- * Hip distance

These measurements create a pose map that acts as a reference for virtual garment placement and scaling.

3. Overlay & Rendering

Using libraries such as OpenCV or WebGL:

- * Clothing images or 3D garments are resized, rotated, and aligned according to the body landmarks.
- * The garment is blended with the live camera feed to create a virtual try-on effect.

Important MediaPipe Modules for Virtual Try-On

MediaPipe Pose

Used for:

- * T-shirts
- * Jackets
- * Pants
- * Full-body outfits

Provides 33 body landmarks.

MediaPipe Hands

Tracks 21 hand landmarks.

Used for:

- * Rings
- * Watches
- * Bracelets
- * Gloves

MediaPipe Face Landmarker

Tracks facial landmarks.

Used for:

- * Glasses
- * Earrings
- * Hats
- * Makeup filters

MediaPipe Holistic

Combines:

- * Pose
- * Face
- * Hand tracking

Used for complete AR/VR virtual try-on systems.

Advantages of MediaPipe Pose

Real-Time Performance

Runs efficiently on:

- * Mobile devices
- * CPUs
- * Web browsers

Low latency enables smooth virtual try-on experiences.

Lightweight Framework

Requires:

- * Low memory
 - * Minimal computational resources
- Suitable for real-time applications.

Cross-Platform Support

Works with:

- * Python
- * JavaScript
- * Android
- * iOS
- * Unity

Useful for WebAR and mobile AR applications.

Technical Challenges

Occlusion Problems

Body parts may become hidden when:

- * Arms cross
 - * User turns sideways
 - * Objects block the body
- This can reduce tracking accuracy.

Cloth Alignment Issues

Loose or folded garments are difficult to align correctly using simple 2D overlays.

Advanced AI cloth simulation may be required for realistic fitting.

Lighting & Background Sensitivity

Poor lighting or cluttered backgrounds can reduce pose detection accuracy.

Conclusion

MediaPipe Pose is an effective framework for building real-time virtual try-on systems because of its speed, lightweight architecture, and cross-platform support. It is widely used in AR/VR fashion applications for body tracking and garment alignment.

OpenPose for Virtual Try-On Systems

OpenPose is a real-time multi-person pose estimation framework developed by Carnegie Mellon University (CMU). It detects human body, face, hand, and foot keypoints from images or videos.

In Virtual Try-On (VTO) systems, OpenPose is used to track body posture and movements so that virtual garments can be aligned accurately with the user's body.

Working Pipeline of OpenPose in VTO

1. Human Detection

The system first detects the presence of one or more people in the camera frame.

It identifies:

- * Body position
- * Orientation
- * Human regions

This helps isolate the user for pose estimation.

2. Keypoint Estimation

OpenPose predicts body landmarks such as:

- * Shoulders
- * Elbows
- * Wrists
- * Hips
- * Knees
- * Ankles

It can detect:

- * 2D body keypoints
- * Hand keypoints
- * Facial landmarks
- * Foot keypoints

The detected points form a digital body skeleton.

3. Pose Mapping

The generated skeleton is used to:

- * Measure body proportions
- * Estimate posture
- * Align virtual garments

This enables clothing placement according to user movement.

4. Garment Overlay & Rendering

Using computer vision libraries such as OpenCV:

- * Clothing assets are resized and rotated
- * Garments are aligned to body landmarks
- * Virtual clothing is rendered on the live video feed

This creates the virtual try-on experience.

Features of OpenPose

Multi-Person Tracking

OpenPose can track multiple users simultaneously in a single frame.

Useful for:

- * Group AR experiences
- * Fashion events
- * Interactive mirrors

Full Body Tracking

Tracks:

- * Body
- * Hands
- * Face
- * Feet

This provides detailed motion understanding.

Real-Time Pose Detection

Capable of detecting poses continuously from video streams.

Supports:

- * Webcam input
- * Recorded video
- * Live camera feeds

Advantages of OpenPose

High Accuracy

Provides detailed and accurate body landmark detection.

Suitable for:

- * Fashion fitting
- * Motion analysis
- * AR applications

Multi-Person Support

Unlike many lightweight frameworks, OpenPose can detect multiple people simultaneously.

Detailed Landmark Detection

Supports:

- * Facial expressions
- * Finger tracking
- * Full-body motion

Improves realism in AR/VR systems.

Limitations of OpenPose

High Computational Cost

Requires:

- * Powerful GPU
- * More memory
- * Higher processing power

Performance may decrease on low-end devices.

Slower Than MediaPipe

OpenPose is generally slower because it performs more complex calculations.

Not ideal for lightweight mobile applications.

Complex Installation

Setup requires:

- * CUDA
- * Deep learning dependencies
- * GPU configuration

Can be difficult for beginners.

Applications of OpenPose in Virtual Try-On

Used for:

- * Virtual clothing fitting
- * AR mirrors
- * Motion tracking
- * Interactive fashion systems
- * Gesture-based interfaces.

Challenges in Virtual Try-On with OpenPose

Occlusion Handling

Tracking accuracy decreases when body parts become hidden.

Real-Time Performance

High-resolution video processing may introduce lag.

Cloth Simulation

Realistic cloth movement still requires additional physics or AI models.

Conclusion

OpenPose is a powerful pose estimation framework widely used in AR/VR and virtual try-on applications. It provides highly accurate body tracking and supports multi-person detection, making it suitable for advanced virtual fashion systems. However, it requires higher computational resources compared to lightweight frameworks such as MediaPipe Pose.

SMPL Body Model for Virtual Try-On Systems

SMPL (Skinned Multi-Person Linear Model) is a 3D parametric human body model used for realistic human body reconstruction and avatar generation. It is widely used in AR/VR, animation, gaming, and virtual try-on systems.

SMPL creates realistic 3D human body shapes using body measurements and pose parameters.

In Virtual Try-On (VTO) systems, SMPL helps generate a personalized 3D avatar on which virtual garments can be fitted accurately.

How SMPL Works in Virtual Try-On

1. Body Shape Estimation

The system estimates the user's body dimensions from:

- * Images
- * Videos
- * Depth sensors
- * AI pose estimation models

Parameters such as:

- * Height
 - * Weight
 - * Shoulder width
 - * Waist size
- are used to generate a body shape.

2. Pose Estimation

Pose estimation models such as:

- * MediaPipe Pose
- * OpenPose

provide body joint positions.

SMPL uses these joint positions to create realistic body movement and posture.

3. 3D Avatar Generation

Using shape and pose parameters:

- * A realistic 3D human mesh is generated
- * The avatar matches the user's body proportions

The model contains:

- * Vertices
- * Skeleton joints
- * Skin deformation information

4. Garment Fitting & Simulation

Virtual clothes are placed on the 3D avatar.

The system performs:

- * Cloth alignment
- * Scaling
- * Collision handling
- * Cloth deformation

This creates a more realistic fitting experience compared to simple 2D overlays.

Components of SMPL

Shape Parameters

Control:

- * Body size
- * Body proportions
- * Physical appearance

Used for personalized avatar generation.

Pose Parameters

Control:

- * Joint rotations
- * Body posture
- * Movement

Used for animation and real-time tracking.

Mesh Representation

SMPL generates a full 3D mesh consisting of:

- * Thousands of vertices
- * Triangular surfaces
- * Skeleton structure

This enables realistic rendering.

Advantages of SMPL in Virtual Try-On

Realistic 3D Representation

Creates accurate human body shapes and movements.

Improves realism in AR/VR applications.

Personalized Fitting

Supports custom avatars based on user body measurements.

Useful for:

- * Fashion fitting
- * Size recommendation
- * Custom tailoring

Animation Support

Works well with:

- * Motion capture
- * Pose estimation
- * Real-time avatar animation

Better Cloth Simulation

Enables realistic:

- * Cloth draping
- * Folding
- * Stretching
- * Body interaction

Applications of SMPL

Used in:

- * Virtual try-on systems
- * Gaming
- * Animation
- * AR/VR experiences
- * Motion capture systems
- * Digital humans

Limitations of SMPL

High Computational Cost

3D body reconstruction requires:

- * GPU acceleration
- * More memory
- * Higher processing power

Complex Integration

Requires knowledge of:

- * 3D graphics
- * Machine learning
- * Mesh processing

Implementation can be difficult for beginners.

Real-Time Challenges

High-quality cloth simulation may reduce rendering speed in real-time applications.

SMPL vs Traditional 2D Overlay Systems

Feature	SMPL	2D Overlay	
-----	-----	-----	
Representation	3D Body Model	Flat 2D Image	
Realism	High	Medium	
Cloth Simulation	Advanced	Limited	
Personalization	High	Low	
Computational Cost	High	Low	

Challenges in SMPL-Based Virtual Try-On

Accurate Body Measurement

Incorrect body estimation can affect avatar accuracy.

Cloth Physics

Realistic cloth movement requires advanced simulation techniques.

Real-Time Rendering

Complex 3D rendering may introduce lag on low-end devices.

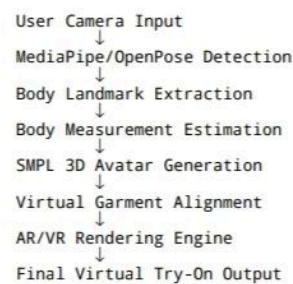
Conclusion

SMPL is a powerful 3D human body model widely used in modern AR/VR virtual try-on systems. It enables realistic avatar creation, accurate garment fitting, and advanced cloth simulation. Although computationally expensive, SMPL provides significantly better realism and personalization compared to traditional 2D virtual try-on approaches.

Comparison Table

Feature	MediaPipe Pose	OpenPose	SMPL
Type	Pose Estimation	Multi-Person Pose Estimation	3D Body Model
Speed	Fast	Moderate	Moderate
Accuracy	High	Very High	Very High
GPU Requirement	Low	High	High
Mobile Support	Excellent	Limited	Limited
Avatar Generation	No	No	Yes
Best Use	Real-time AR Try-On	Advanced Tracking	3D Virtual Fitting

Architecture of AI-Based AR/VR Virtual Try-On System



Working demo of mediapipe:

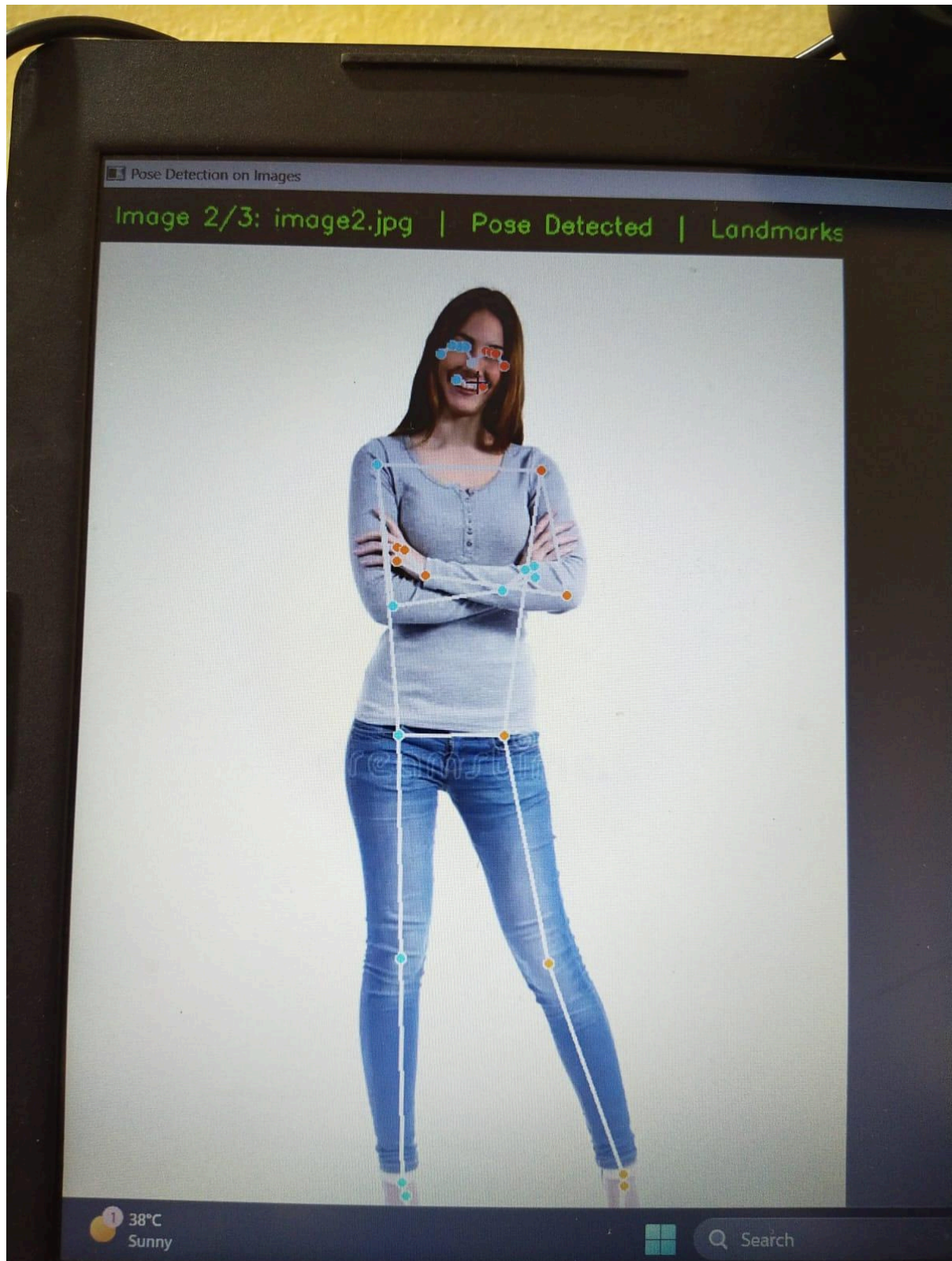


Figure 1: MediaPipe Pose Landmark Detection

The image demonstrates real-time human pose estimation using MediaPipe Pose. The framework detects body landmarks and generates a skeletal structure for body tracking, posture analysis, and virtual garment alignment in AR/VR virtual try-on systems.

Pose Measurements & Recommendation

Shoulder Width: 40.00 cm

Hip Width: 25.09 cm

Left Upper Arm: 31.34 cm

Right Upper Arm: 34.50 cm

Left Forearm: 34.53 cm

Right Forearm: 28.83 cm

Right Thigh: 52.10 cm

Right Leg: 50.83 cm

Torso Length: 65.48 cm

Neck Length: 31.30 cm

Shirt: M | Pant: 32



Figure 2: AI-Based Body Measurement and Size Recommendation

The image shows AI-based body measurement estimation using MediaPipe Pose landmarks. Measurements such as shoulder width, torso length, and arm length are calculated to support virtual fitting and automatic clothing size recommendation in virtual try-on applications.

Code :

```
import cv2
import mediapipe as mp
import math
import sys

# — MediaPipe Setup


---


mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
mp_styles = mp.solutions.drawing_styles

# — Pixel Distance


---


def dist(p1, p2, w, h):
    x1, y1 = p1.x * w, p1.y * h
    x2, y2 = p2.x * w, p2.y * h
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

# — Size Recommendation


---


def get_shirt_size(shoulder_cm):
    if shoulder_cm < 36: return 'XS'
    elif shoulder_cm < 39: return 'S'
    elif shoulder_cm < 42: return 'M'
    elif shoulder_cm < 45: return 'L'
    elif shoulder_cm < 48: return 'XL'
    else: return 'XXL'

def get_pant_size(hip_cm):
    if hip_cm < 20: return '26'
    elif hip_cm < 22: return '28'
    elif hip_cm < 25: return '30'
    elif hip_cm < 28: return '32'
    elif hip_cm < 31: return '34'
    else: return '36'
```

— Draw colored connection

```
def draw_line(img, p1, p2, w, h, color=(0, 120, 255), thickness=2):
    x1, y1 = int(p1.x * w), int(p1.y * h)
    x2, y2 = int(p2.x * w), int(p2.y * h)
    cv2.line(img, (x1, y1), (x2, y2), color, thickness)
```

— Process Image

```
def process_image(image_path):
    image = cv2.imread(image_path)
    if image is None:
        print(f"Could not read image: {image_path}")
        return

    # Resize if too large
    h, w = image.shape[:2]
    if w > 1000:
        scale = 1000 / w
        image = cv2.resize(image, (1000, int(h * scale)))
        h, w = image.shape[:2]

    with mp_pose.Pose(
        model_complexity=1,
        smooth_landmarks=True,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5
    ) as pose:

        rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        rgb.flags.writeable = False
        results = pose.process(rgb)
        rgb.flags.writeable = True

        if not results.pose_landmarks:
            print("No pose detected!")
            cv2.putText(image, "No Pose Detected", (10, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)
            cv2.imshow("Pose Measurements", image)
            cv2.waitKey(0)
            return
        lm = results.pose_landmarks.landmark

    # — Scale factor
```

```
# Use shoulder width as reference (avg = 40cm)
shoulder_px = dist(lm[11], lm[12], w, h)
```



```
scale = 40.0 / shoulder_px if shoulder_px > 0 else 0.1
```

```
# — Calculate all measurements
```

```
shoulder_cm = dist(lm[11], lm[12], w, h) * scale
hip_cm      = dist(lm[23], lm[24], w, h) * scale
left_upper_cm = dist(lm[11], lm[13], w, h) * scale
right_upper_cm = dist(lm[12], lm[14], w, h) * scale
left_fore_cm = dist(lm[13], lm[15], w, h) * scale
right_fore_cm = dist(lm[14], lm[16], w, h) * scale
left_thigh_cm = dist(lm[23], lm[25], w, h) * scale
right_thigh_cm = dist(lm[24], lm[26], w, h) * scale
left_leg_cm = dist(lm[25], lm[27], w, h) * scale
right_leg_cm = dist(lm[26], lm[28], w, h) * scale
torso_cm     = ((dist(lm[11], lm[23], w, h) +
                  dist(lm[12], lm[24], w, h)) / 2) * scale
neck_cm      = dist(lm[0], lm[11], w, h) * scale
```

```
# — Size recommendations
```

```
shirt = get_shirt_size(shoulder_cm)
pant = get_pant_size(hip_cm)
```

```
# — Draw colored skeleton
```

```
BLUE = (255, 100, 0)
WHITE = (255, 255, 255)
```

```
# Torso
```

```
draw_line(image, lm[11], lm[12], w, h, BLUE, 2)
draw_line(image, lm[11], lm[23], w, h, BLUE, 2)
draw_line(image, lm[12], lm[24], w, h, BLUE, 2)
draw_line(image, lm[23], lm[24], w, h, BLUE, 2)
```

```
# Arms
```

```
draw_line(image, lm[11], lm[13], w, h, BLUE, 2)
draw_line(image, lm[13], lm[15], w, h, BLUE, 2)
draw_line(image, lm[12], lm[14], w, h, BLUE, 2)
draw_line(image, lm[14], lm[16], w, h, BLUE, 2)
```

```
# Legs
```

```
draw_line(image, lm[23], lm[25], w, h, BLUE, 2)
draw_line(image, lm[25], lm[27], w, h, BLUE, 2)
draw_line(image, lm[24], lm[26], w, h, BLUE, 2)
draw_line(image, lm[26], lm[28], w, h, BLUE, 2)
```

```
# Height line (white)
```

```
nose_x = int(lm[0].x * w)
```

```

nose_y = int(lm[0].y * h)
ank_y = int(((lm[27].y + lm[28].y) / 2) * h)
cv2.line(image, (nose_x - 20, nose_y),
          (nose_x - 20, ank_y), WHITE, 2)

```

— Draw landmark dots with numbers

```

key_lm = [0,11,12,13,14,15,16,23,24,25,26,27,28]
for i in key_lm:
    px = int(lm[i].x * w)
    py = int(lm[i].y * h)
    cv2.circle(image, (px, py), 6, (0, 0, 255), -1)
    cv2.circle(image, (px, py), 2, (255,255,255), -1)
    cv2.putText(image, str(i), (px+4, py-4),
                cv2.FONT_HERSHEY_SIMPLEX, 0.35, (0, 255, 255), 1)

```

— Measurements text panel

```

measurements = [
    f"Shoulder Width: {shoulder_cm:.2f} cm",
    f"Hip Width: {hip_cm:.2f} cm",
    f"Left Upper Arm: {left_upper_cm:.2f} cm",
    f"Right Upper Arm: {right_upper_cm:.2f} cm",
    f"Left Forearm: {left_fore_cm:.2f} cm",
    f"Right Forearm: {right_fore_cm:.2f} cm",
    f"Right Thigh: {right_thigh_cm:.2f} cm",
    f"Right Leg: {right_leg_cm:.2f} cm",
    f"Torso Length: {torso_cm:.2f} cm",
    f"Neck Length: {neck_cm:.2f} cm",
]

```

```

y_start = 30
for i, text in enumerate(measurements):
    cv2.putText(image, text, (10, y_start + i * 28),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55,
                (0, 255, 255), 2)

```

— Size recommendation

```

cv2.putText(image,
    f"Shirt: {shirt} | Pant: {pant}",
    (10, y_start + len(measurements) * 28 + 10),
    cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)
output_path = image_path.replace('.jpg','_output.jpg').replace('.png','_output.png')
cv2.imwrite(output_path, image)
print(f"Output saved: {output_path}")
print(f"\n=== MEASUREMENTS ===")
for m in measurements:

```

```

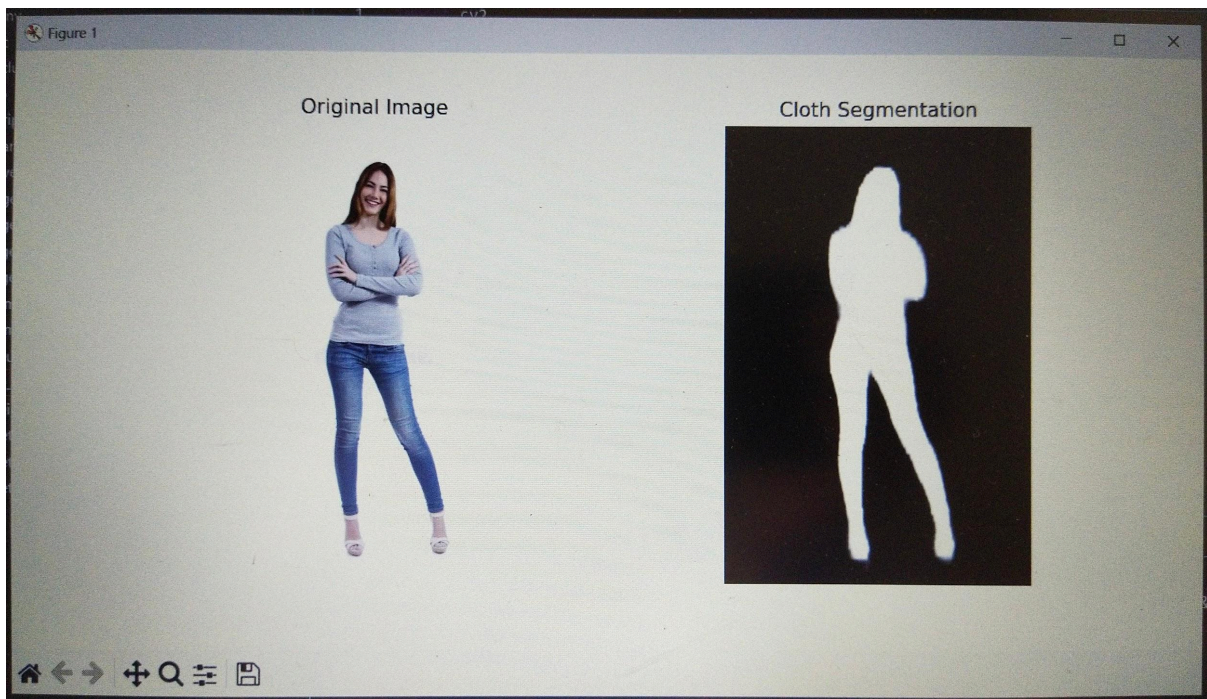
        print(" ", m)
        print(f"\n Shirt Size : {shirt}")
        print(f" Pant Size : {pant}")

        cv2.imshow("Pose Measurements & Recommendation", image)
        print("\nPress any key to close...")
        cv2.waitKey(0)
        cv2.destroyAllWindows()
if __name__ == "__main__":
    IMAGE_PATH = "C:/Users/dell/Desktop/aiml/image1.jpg"
    process_image(IMAGE_PATH)

```

Segmentation:

Figure 3:



Code:

```

# Load MediaPipe segmentation model
mp_selfie_segmentation = mp.solutions.selfie_segmentation
segment = mp_selfie_segmentation.SelfieSegmentation(
    model_selection=1
)
# Read image
img = cv2.imread("person.jpg")
# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# Process segmentation
results = segment.process(img_rgb)
# Get mask
mask = results.segmentation_mask

```

```

# Display outputs
plt.figure(figsize=(10,5))
# Original image
plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
# Segmentation mask
plt.subplot(1,2,2)
plt.imshow(mask, cmap='gray')
plt.title("Cloth Segmentation")
plt.axis('off')
plt.show()

```

Integration Possibilities for Raritone

1. Camera captures user image or video
2. MediaPipe/OpenPose detects body landmarks
3. AI estimates body measurements
4. SMPL generates 3D avatar
5. Virtual garments are aligned with the body
6. AR engine renders final virtual try-on output

Challenges Faced:

- Accurate body measurement estimation
- Realistic garment fitting
- Occlusion handling
- Lighting variation issues
- Real-time rendering optimization
- High GPU requirements for advanced 3D simulation

Future Improvements:

- AI-based personalized size recommendation
- Realistic cloth physics simulation
- Full-body 3D avatar customization
- Multi-angle virtual try-on
- WebAR support
- VR fashion showroom integration

Conclusion:

The research explored AI-based technologies used in AR/VR virtual try-on systems. MediaPipe Pose, OpenPose, SMPL body models, body measurement systems, and segmentation techniques play an important role in realistic virtual fashion experiences. These technologies can help Raritone develop advanced and interactive virtual try-on solutions.